

SOFTWARE EMBARQUE

FOX G-20



TP

Encadré par :
Pr. AICHA ALAMI HASSANI

Realisé par:
FATIMA ZAHRA AIT SKOURT
OMAR ALAOUI MABROOK BUFTEEM
MOHAMED ABAlJI

L'objectif du TP :

Le but de ce TP était de développer un système communicant via une liaison série et de l'évoluer en intégrant une communication avec un serveur Web embarqué. En utilisant des commandes gcc, l'objectif était de créer des programmes pour la FOX G20, en se concentrant sur la compréhension de l'architecture des GPIO sous Linux. En pratique, cela impliquait la mise en œuvre de divers programmes pour contrôler des éléments tels que des LED et un compteur 7 segments sur la maquette IDL_800A_DIGITAL_LAB, combinant ainsi la manipulation des GPIO avec des fonctions communicantes pour atteindre des interactions plus avancées et polyvalentes.

I. Présentation du matériel :

1. La platine FOX Board G20

La platine "FOX Board G20" est un système embarqué avec un OS Linux, construite autour d'un processeur ARM9™ AT91SAM9G20 cadencé à 400 MHz d'Atmel™.

Elle est composée d'une platine support avec étage de régulation multiple (2 x 3,3 V / 1,8 V / 1 V) sur laquelle est insérée le module CPU « NETUS G20 » (lequel intègre le processeur AT91SAM9G20). La platine support fournit :

- Un connecteur d'alimentation
- Un connecteur Ethernet (Base 10/100)
- 2 ports USB 2.0 host
- Un port client sur mini USB
- Une pile de sauvegarde pour horloge RTC,
- Un emplacement pour afficheurs uOLED intelligents de "4D Systems"
- Un connecteur mâle au pas de 2,54 mm pour le raccordement du port série "console/Debug"
- Une zone de prototypage rapide disposant de ports série, ports d'entrées/sorties parallèle, I2C/SPI, convertisseurs A/N, PWM, ...

Module NETUS G20

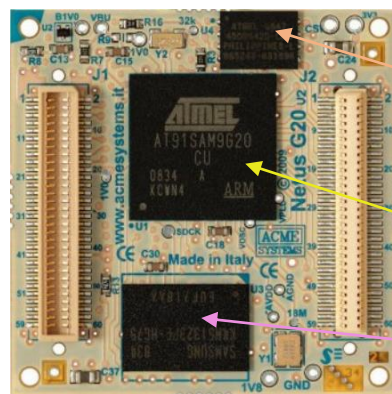
Ports USB 12 MB/s

Zone de
prototypage
Connecteur

5VDC power-in

Port de débogage

Zone de
prototypage
Connecteur



DATAFLASH

CPU

RAM

2. Maquette IDL_800A_DIGITAL_LAB :

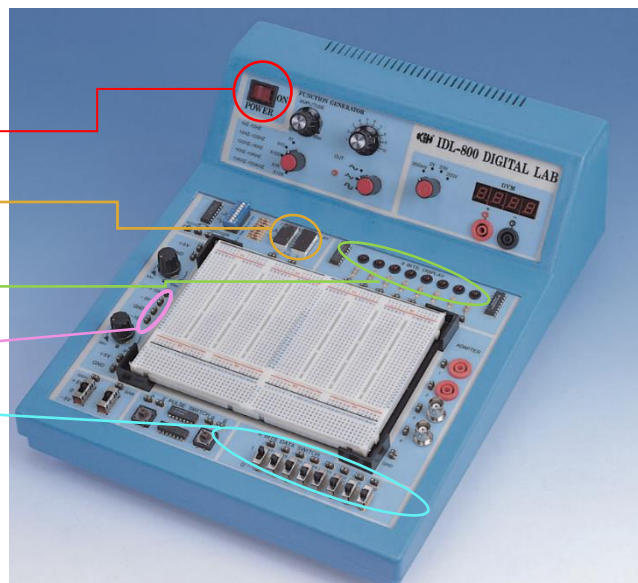
Power button

Afficheurs 7 segments

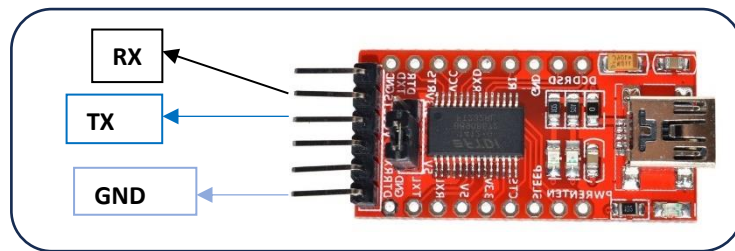
LEDs (Chenard)

5V / GND

Switches



3. FTDI 232 :



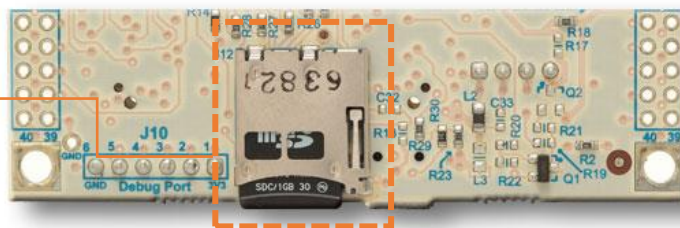
4. Cable Ethernet & Cable USB :



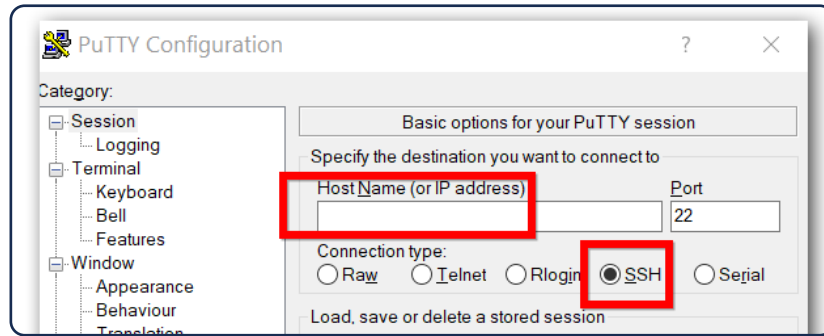
II. Branchements :

Carte micro SD contenant deux partitions (Capacité maximale de 8 Go) :

- **FAT16 : Noyau (Kernel)**
- **EXT4 : système de fichiers.**



- Pour la transmission, la réception et la conversion de données, on a branché la carte FOX G20 avec FTDI232, en liant GND à GND, RX du FTDI à TX du FOX G20 et vice versa, tout ça en utilisant le port de débogage du FOX G20.
- Pour l'alimentation on a utilisé le port 5VDC, en connectant 5V du maquette IDL-800A avec 5V de la carte FOX, et GND de la maquette à GND de FOX G20.
- Après vérifié le montage par l'encadrante, on a connecté le FTDI au PC par le câble USB, après avoir installé CMD21216_setup et PUTTY, le premier s'agit d'un driver (pilote) utilisé pour permettre un périphérique comme FTDI dans notre cas, d'être correctement reconnu par le logiciel PUTTY, le deuxième est un logiciel client léger et open-source utilisé pour établir des connexions réseau sécurisées, principalement pour se connecter à des appareils distants via des protocoles bien définis.
- Puis, on a défini le port, le débit et le type de connexion(série) dans le logiciel :



III. Programmation en ligne de commandes (Shell) :

En alimentant la maquette et démarrant la carte FOX G20, on nous demande de s'identifier après l'ouverture de la fenêtre de commande :

Login : root

Password : netusg20

Avant de se connecter au site web, on a essayé d'allumer une LED et de lire la valeur d'un switch en exécutant les commandes suivantes :

Premièrement on a allumé la LED, en utilisant la ligne 9 pin J6.26 :

echo 9 >/sys/class/gpio/export (pour l'activation de la ligne)

echo out >/sys/class/gpio/pioA9/direction (pour la considérer comme sortie)

echo 1 >/sys/class/gpio/pioA9/value (pour passer à l'état haut (allumage de la LED))

Deuxièmement on a configuré un switch comme étant une entrée, en utilisant la ligne 50 pin J7.3 :

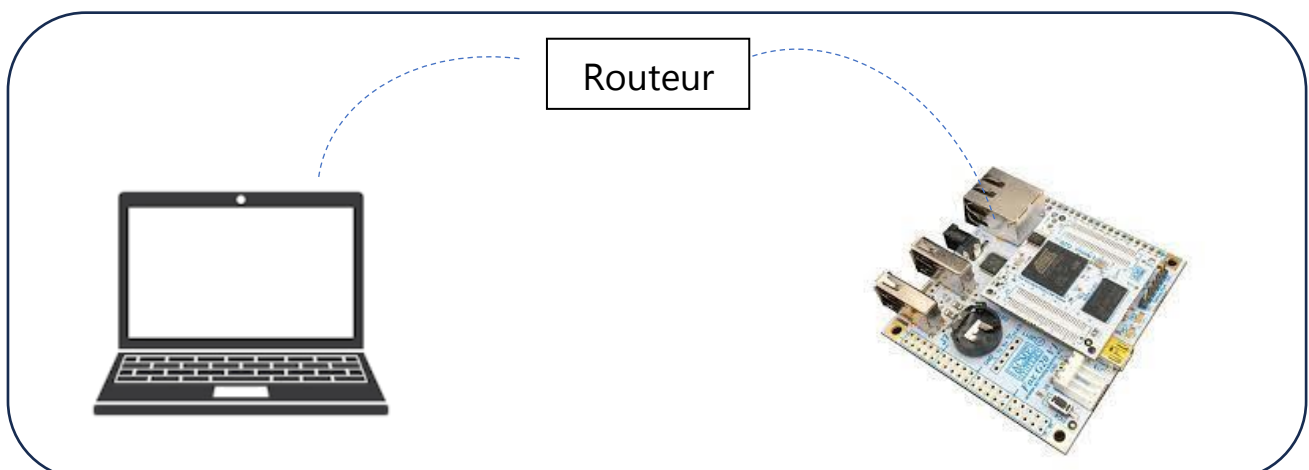
#echo 50 > / sys / class / gpio / export

#echo IN > /sys / class / gpio / pioB18/ direction

cat /sys/class/gpio/pioB18/value (pour lire l'état de la ligne)

IV. Programmation en C :

D'abord, il faut connecter la carte FOX G20 à un routeur à l'aide d'un câble Ethernet, et se connecter au réseau qu'on a créé à partir du PC :



Après cette étape, on utilise la commande « *\$ifconfig eth0* » pour obtenir une adresse IP, comme on peut forcer une adresse précise par la commande « *\$ifconfig eth0 192.168.2.109* », on insère notre adresse dans un navigateur web (‘ Edge’ dans notre cas).

Une page s’affiche contenant deux environnements : le Shell et le Codiad, en choisissant Codiad, on se connecte avec le login et le mot de passe indiqué et on se trouve dans le Workspace, puis on commence la création d’un nouveau projet contenant des fichiers sources programmés en C.

Les fonctions nécessaires prédéfini :

```
#include <string.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <fcntl.h>
#include <time.h>
#define LOW 0
#define HIGH 1
#define IN 1
#define OUT 0
struct S_GPIO_LINE { char id_line[5];
int id_pin;
int direction;
int value;
};
typedef struct S_GPIO_LINE S_GPIO_LINE;
/* ***** */
/* LOAD_GPIO_LINE */
/* Initialisation of a GPIO line */
/* ***** */
int load_gpio_line(S_GPIO_LINE *ps_line ) {
FILE *p_gpio_line;
/* Exporting GPIO line */
if ((p_gpio_line = fopen("/sys/class/gpio/export", "w")) == NULL) {
printf("Can not open export file.\n");
exit(1);
}
fprintf(p_gpio_line,"%d",ps_line->id_pin);
fclose(p_gpio_line);
return 0; }

/* ***** */
/* SET_GPIO_DIRECTION */
/* Sets the direction (IN/OUT) of a GPIO line */ /* ***** */
int set_gpio_direction(S_GPIO_LINE *ps_line, int i_direction) {
int fd; char path[35]; //file//chemi
n
snprintf(path,35, "/sys/class/gpio/pio%s/direction",ps_line->id_line); printf("path:%s\n",path);
```

```

fd = open(path, O_WRONLY); if (-1 == fd) {
fprintf(stderr, "Failed to open gpio direction for writing!\n");
exit(1);
} if( i_direction ) {
if (-1 == write(fd, "in", 36)) {
fprintf(stderr, "Failed to set direction!\n"); exit(1);
} } else {
if (-1 == write(fd, "out", 36)) {
fprintf(stderr, "Failed to set direction!\n"); exit(1);
} }
return 0; }
/* ***** */
/* SET_GPIO_LINE */
/* Sets the value of a GPIO line */
/* ***** */
int set_gpio_line(S_GPIO_LINE *ps_line, int value) {
FILE *p_gpio_value;
char gpio_value[35];
char c_value[2];
if( 1 ) {
sprintf(gpio_value, "/sys/class/gpio/pio%s/value", ps_line->id_line);
/* Setting value */
if ((p_gpio_value = fopen(gpio_value, "w")) == NULL)
{
printf("Cannot open value file.\n");
exit(1);
}
rewind(p_gpio_value);
sprintf(c_value, "%d", value);
ps_line->value = value;
fwrite(&c_value, sizeof(char), 1, p_gpio_value);
fclose(p_gpio_value);
} else{
printf("Wrong access.\n");
exit(1);
} return 0;
}
/* ***** */
/* GET_GPIO_LINE */
/* Gets value of a GPIO line */
/* ***** */
int get_gpio_line(S_GPIO_LINE *ps_line) {
FILE *p_gpio_value;
char gpio_value[35];
int value = 0;
if( 1 ) {
sprintf(gpio_value, "/sys/class/gpio/pio%s/value", ps_line->id_line);
printf("**");
/* Getting value */
if ((p_gpio_value = fopen(gpio_value, "r")) == NULL) {
printf("Cannot open value file.\n");
exit(1);
}

```

```

}
value = fgetc (p_gpio_value) - 48; fclose(p_gpio_value);
} else {
printf("Wrong access.\n");
exit(1);
}
return value;
}

```

1 : LED contrôlé par un SWITCH.

```

int main() {
//configuration LED
S_GPIO_LINE LED;
LED.id_pin=28; //J7.25
sprintf(LED.id_line, "A28");
load_gpio_line(&LED); set_gpio_direction(&LED, 0);

//configuration SWITCH
S_GPIO_LINE switch;
switx.id_pin=27; //J7.36
sprintf(switx.id_line, "A27");
load_gpio_line(&switch); set_gpio_direction(&switx, 1);

while(1)
{
    if(get_gpio_line(&switch)==1)
        set_gpio_line(&LED, 1);
    else;
        set_gpio_line(&LED, 0);} return 0;
}
}

```

2 : Ce compteur fonctionne de la manière suivante : lorsque l'interrupteur est positionné sur 1, le compteur va de 0 à 9. En revanche, lorsqu'il est sur 0, le compteur décompte de 9 à 0. Il est important de noter que si l'on change la position de l'interrupteur, le compteur modifiera son état après avoir terminé le comptage ou le décomptage en cours.

```

int main() {

    int tabA[10] = {0, 1, 0, 1, 0, 1, 0, 1, 0, 1};
    int tabB[10] = {0, 0, 1, 1, 0, 0, 1, 1, 0, 0};
    int tabC[10] = {0, 0, 0, 0, 1, 1, 1, 1, 0, 0};
    int tabD[10] = {0, 0, 0, 0, 0, 0, 0, 0, 1, 1};

    S_GPIO_LINE L[4], sw;
    int i,j;

    L[0].id_pin = 34;
    L[1].id_pin = 33;
    L[2].id_pin = 35;
    L[3].id_pin = 48;
    sw.id_pin = 53;

    sprintf(L[0].id_line, "B2");

```



```

sprintf(L[1].id_line, "B1");
sprintf(L[2].id_line, "B3");
sprintf(L[3].id_line, "B16");
sprintf(sw.id_line, "B21");

load_gpio_line(&sw);
set_gpio_direction(&sw, 1);

for (j = 0; j < 4; j++)
{
    load_gpio_line(&L[j]);
    set_gpio_direction(&L[j], 0);
}

while (1)
{
    get_gpio_line(&sw);

    if(sw.value==1)
    {
        for (i = 0; i < 10; i++)
        {
            set_gpio_line(&L[0], tabA[i]);
            set_gpio_line(&L[1], tabB[i]);
            set_gpio_line(&L[2], tabC[i]);
            set_gpio_line(&L[3], tabD[i]);
            sleep(1);
        }
    }
    else{
        for (i = 9; i >= 0; i--)
        {
            set_gpio_line(&L[0], tabA[i]);
            set_gpio_line(&L[1], tabB[i]);
            set_gpio_line(&L[2], tabC[i]);
            set_gpio_line(&L[3], tabD[i]);
            sleep(1);
        }
    }
    return 0;
}

```

3 : Le compteur alternera entre deux états (comptage et décomptage) en fonction du changement de l'interrupteur SW2, tandis que l'interrupteur SW1 contrôlera le fonctionnement du compteur : 1 pour activer et 0 pour désactiver.

```

int main() {

    int tabA[10] = {0, 1, 0, 1, 0, 1, 0, 1, 0, 1};
    int tabB[10] = {0, 0, 1, 1, 0, 0, 1, 1, 0, 0};
    int tabC[10] = {0, 0, 0, 0, 1, 1, 1, 1, 0, 0};
    int tabD[10] = {0, 0, 0, 0, 0, 0, 0, 0, 1, 1};

    S_GPIO_LINE L[4], sw2,sw1;
    int i,j;

    L[0].id_pin = 34;

```

```

L[1].id_pin = 33;
L[2].id_pin = 35;
L[3].id_pin = 48;
sw2.id_pin = 53;
sw1.id_pin = 52;

sprintf(L[0].id_line, "B2");
sprintf(L[1].id_line, "B1");
sprintf(L[2].id_line, "B3");
sprintf(L[3].id_line, "B16");
sprintf(sw2.id_line, "B21");
sprintf(sw1.id_line, "B20");

load_gpio_line(&sw2);
set_gpio_direction(&sw2, 1);
load_gpio_line(&sw1);
set_gpio_direction(&sw1, 1);

for (j = 0; j < 4; j++)
{
    load_gpio_line(&L[j]);
    set_gpio_direction(&L[j], 0);
}

while (1)
{
    get_gpio_line(&sw1);
    if(sw1.value==1)
    {
        while(1)
        {
            get_gpio_line(&sw2);
            if(sw2.value==1)
            {
                while(1)
                {
                    for (i = 0; i < 10; i++)
                    {
                        set_gpio_line(&L[0], tabA[i]);
                        set_gpio_line(&L[1], tabB[i]);
                        set_gpio_line(&L[2], tabC[i]);
                        set_gpio_line(&L[3], tabD[i]);

                        if(sw2.value==0 || sw1.value==0)
                        break;
                        sleep(1);
                    }
                }
            }
            if(sw2.value==0)
            {
                while(1)
                {
                    for (i = 9; i >= 0; i--)
                    {
                        set_gpio_line(&L[0], tabA[i]);
                        set_gpio_line(&L[1], tabB[i]);
                        set_gpio_line(&L[2], tabC[i]);
                        set_gpio_line(&L[3], tabD[i]);
                        sleep(1);
                        if(sw2.value==1 || sw1.value==0)
                        break;
                    }
                }
            }
            if(sw1.value==0)
            break;
        }
    }
    if(sw1.value==0)
    {
        set_gpio_line(&L[0], 0);
        set_gpio_line(&L[1], 0);
    }
}

```

```

        set_gpio_line(&L[2], 0);
        set_gpio_line(&L[3], 0);
    }
}

return 0;
}

```

Un chenillard : premier interrupteur : Permet d'activer ou de désactiver le fonctionnement des LEDs, deuxième interrupteur : Change le sens de fonctionnement des LEDs.

```
int main() {
    int tabA[4] = {1, 0, 0, 0};
    int tabB[4] = {0, 1, 0, 0};
    int tabC[4] = {0, 0, 1, 0};
    int tabD[4] = {0, 0, 0, 1};

    S_GPIO_LINE L[4], sw2, sw1;
    int i, j;

    L[0].id_pin = 34;
    L[1].id_pin = 33;
    L[2].id_pin = 35;
    L[3].id_pin = 48;
    sw2.id_pin = 53;
    sw1.id_pin = 52;

    sprintf(L[0].id_line, "B2");
    sprintf(L[1].id_line, "B1");
    sprintf(L[2].id_line, "B3");
    sprintf(L[3].id_line, "B16");
    sprintf(sw2.id_line, "B21");
    sprintf(sw1.id_line, "B20");

    load_gpio_line(&sw2);
    set_gpio_direction(&sw2, 1);
    load_gpio_line(&sw1);
    set_gpio_direction(&sw1, 1);

    for (j = 0; j < 4; j++)
    {
        load_gpio_line(&L[j]);
        set_gpio_direction(&L[j], 0);
    }

    while (1)
    {
        get_gpio_line(&sw1);
        if (sw1.value==1)
        {
            while(1)
            {
                get_gpio_line(&sw2);
                if (sw2.value==1)
                {
                    while(1)
                    {
                        for (i = 0; i < 4; i++)
                        {
                            set_gpio_line(&L[0], tabA[i]);
                            set_gpio_line(&L[1], tabB[i]);
                            set_gpio_line(&L[2], tabC[i]);
                            set_gpio_line(&L[3], tabD[i]);
                        }
                    }
                }
            }
        }
    }
}
```

```

        if(sw2.value==0 || sw1.value==0)
            break;
        sleep(1);
    }

    }

    if(sw2.value==0)
    {
        while(1)
        {
            for (i = 4; i > 0; i--)
            {
                set_gpio_line(&L[0], tabA[i]);
                set_gpio_line(&L[1], tabB[i]);
                set_gpio_line(&L[2], tabC[i]);
                set_gpio_line(&L[3], tabD[i]);
                sleep(1);
                if(sw2.value==1 || sw1.value==0)
                    break;
            }
        }
    }

    if(sw1.value==0)
        break;
}

if(sw1.value==0)
{
    set_gpio_line(&L[0], 0);
    set_gpio_line(&L[1], 0);
    set_gpio_line(&L[2], 0);
    set_gpio_line(&L[3], 0);
}

}

return 0;
}

```